

Analysis of IoT Botnet Structure and Behavior: A Study Based on the Hitori (SlumX) Variant

Ariel Matias Melo Pazmiño

Universidad Internacional del Ecuador - Facultad de Ciencias Técnicas - Escuela de Ciencias de la Computación

SlumX: 12 Architectures • 7 Attack Vectors • 1 Sophisticated Threat

The Evolution from Simple Mirai Variants to Advanced Multi-Platform IoT Weaponry

1. Abstract

- Static code analysis of **SlumX botnet** (Hitori variant) demonstrates a fully functional C2 mechanism [1]
- Multiple methods of executing **DDoS attacks** (UDP, TCP and HTTP)
- Supports various device architectures: **MIPS, ARM, x86**, plus nine additional [2]
- Advanced **evasion techniques** specific to IoT devices

Contribution: Detailed description of latest IoT botnet generations, behaviours and mitigation strategies [5].

2. Research Objectives

General Objective

To explore the risk presented by IoT botnets, a detailed technical analysis of the Hitori (SlumX) botnet variant is required [1].

Specific Objectives

- Evaluate the source code to understand how the botnet operates and its main functions.
- Assess DDoS attack techniques and payload delivery methods [3].
- Evaluate multi-architecture and cross-platform support mechanisms.
- Recommend mitigation methods for identified vulnerabilities [4].

3. Methodology

Phase	Description
Data Collection	GitHub repo <code>mat1520/slumpx</code>
Static Analysis	<code>bot.c</code> , <code>server.c</code> , <code>cc.py</code>
Control Flow	Execution paths mapping
Threat Modeling	Attack surface assessment

Note: Static analysis only - no runtime execution [2].

4. Threat Model

12+

Supported Architectures

7+

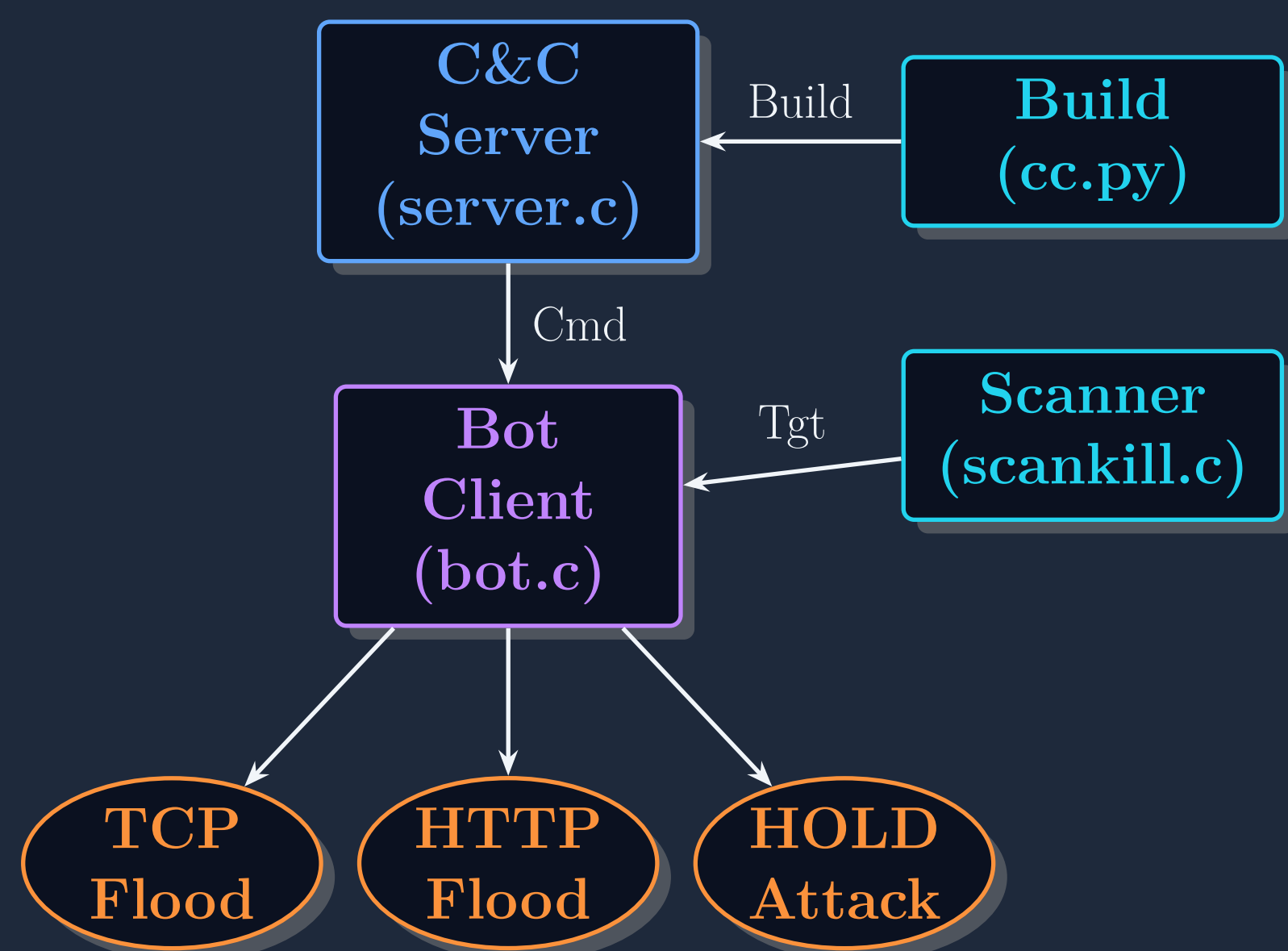
Attack Vectors

1Tbps+

DDoS Potential

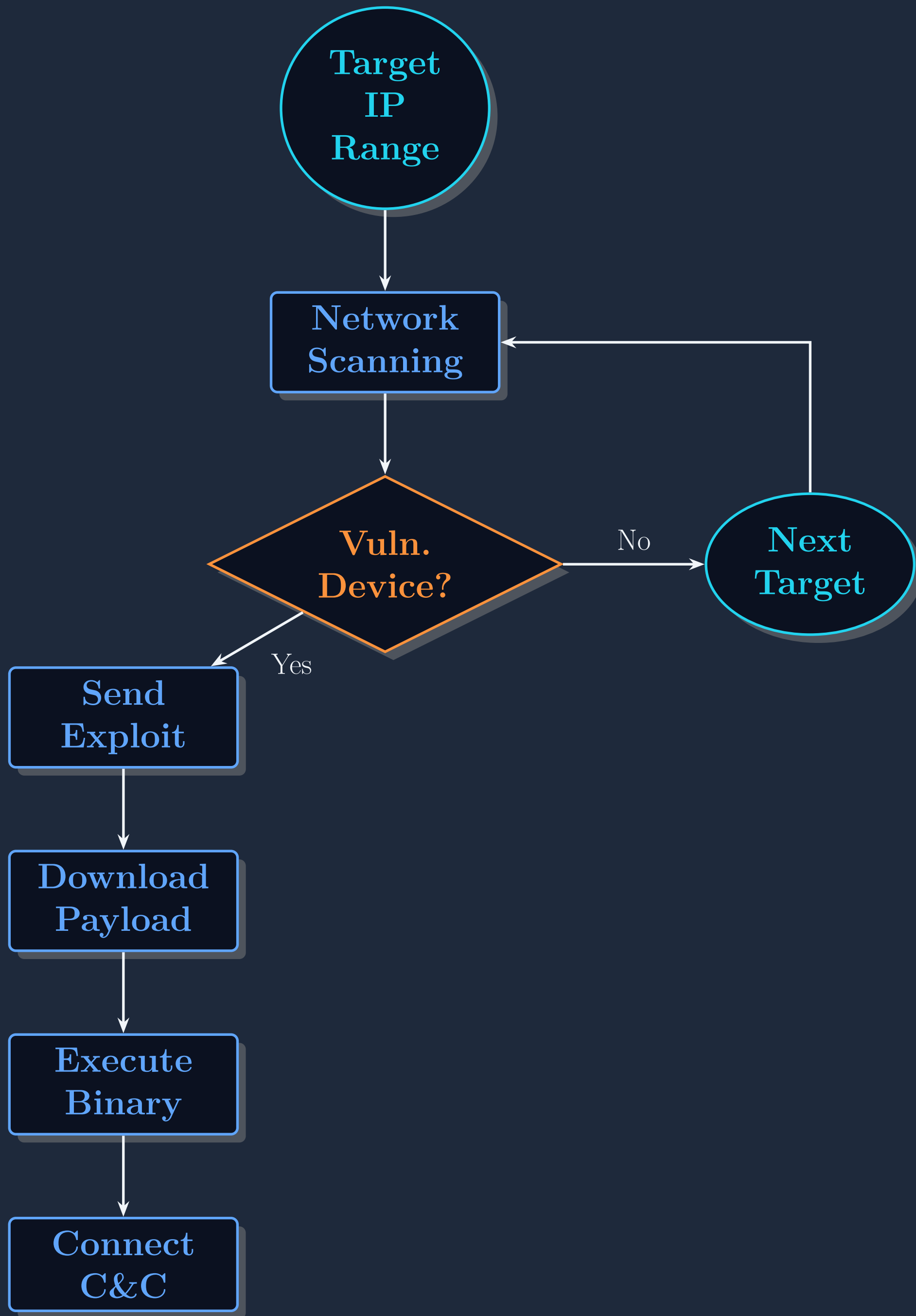
Vulnerabilities: Buffer overflows, no thread sync, hardcoded credentials.

5. System Architecture



A centralized command-and-control model is employed. The server part uses TCP port 1994 to control compromised systems; the `bot.c` component is distributed among infected devices. A Python-based build system (`cc.py`) enables cross-compilation for each processor type.

6. Infection Process Flow



The network scanning phase (`scankill.c`) identifies machines that can potentially be infected. After gaining access (e.g., through Telnet brute force or exploiting CVE vulnerabilities), the payload file is downloaded and executed to establish a continual connection back to the C2 server [2].

7. Security Risk Assessment

Hardcoded C&C (Single Point of Failure)	Plain Text Communication (Traffic Analysis)
Fixed Port Usage (1994) (Filtering)	Predictable Binary Names (Signatures)

High Risk: Hard-coded IP addresses allow defenders to quickly track the entire botnet down (sinkhole). **Medium Risk:** Unencrypted traffic can be easily inspected and signatures developed.

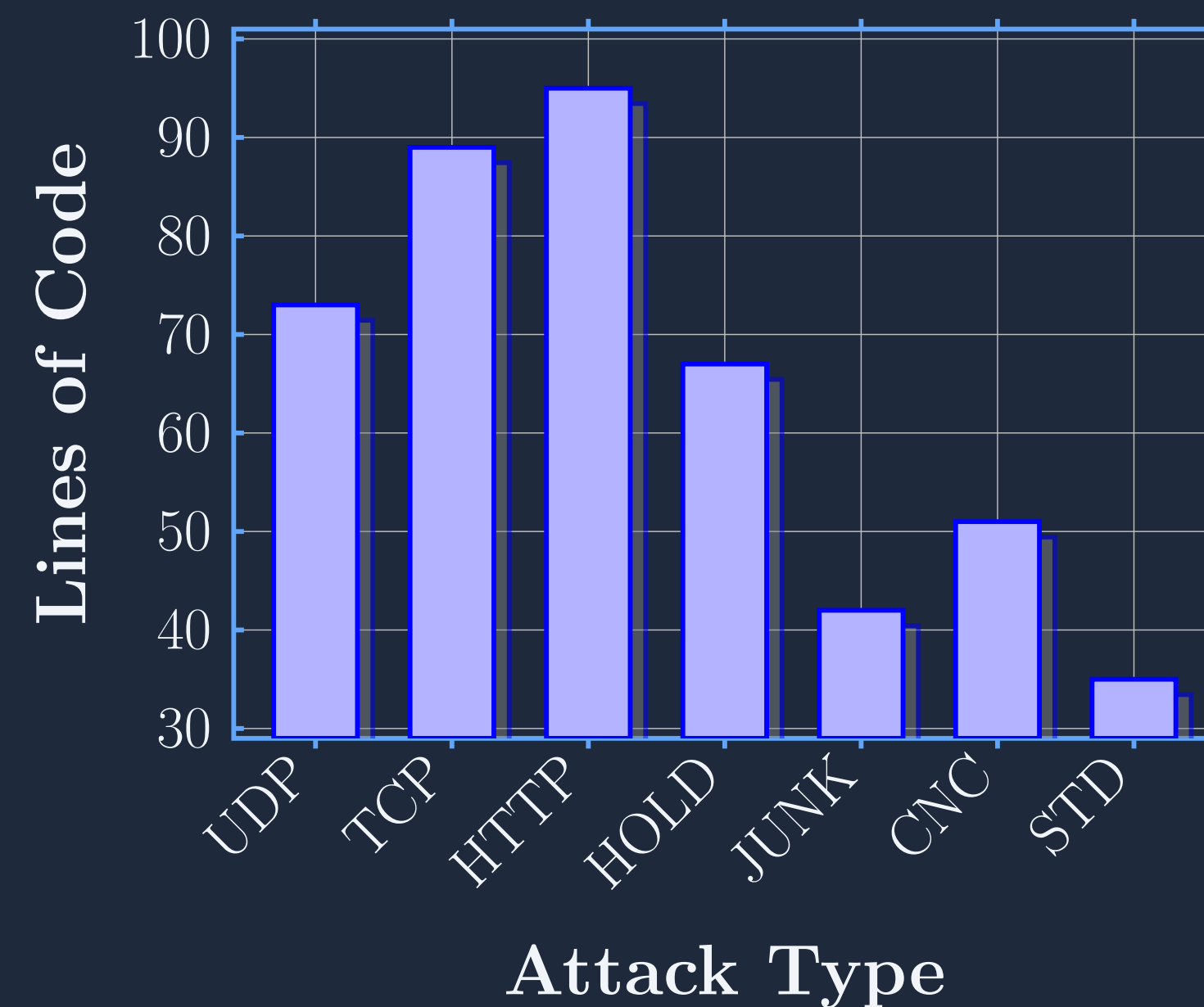
8. Multi-Architecture Support

Architecture	Binary	Target Devices
MIPS	8mips8	Router firmware, embedded systems
MIPSEL	8mps18	Little-endian MIPS devices
SH4	8sh48	SuperH architecture devices
i586	8i5868	Legacy x86 systems
ARMv6l	8arm68	ARM-based IoT devices
i686	8i6868	Modern x86 systems
PowerPC	8ppc8	PowerPC-based systems
M68k	8m68k8	Motorola 68000 systems
SPARC	8spc8	SPARC architecture systems
ARMv4l	8arm48	Older ARM devices
ARMv5l	8arm58	ARMv5 systems
ARMv7l	8arm78	Modern ARM devices

12 architectures supported - Massive IoT attack surface

In contrast to earlier IoT botnets, SlumX has 12 separate device types (architectures) which increases the number of possible infected devices [3][5].

9. Attack Vector Complexity



HTTP-based attacks show highest complexity (95 LOC).

The botnet implements a versatile range of DDoS capabilities. Our code analysis reveals that HTTP flooding represents the most sophisticated vector (95 LOC), requiring the construction of valid headers to bypass standard filters. In contrast, UDP and TCP floods rely on direct raw socket manipulation to generate high-volume volumetric traffic [3].

10. Key Findings & Conclusions

- Multiple Architectures:** Extensive cross-platform support covering twelve distinct processor architectures, including MIPS, ARM, and x86, maximizing the potential infection pool.
- Advanced Evasion:** Implementation of IoT-centric defensive mechanisms, including anti-debugging routines and dynamic process masquerading.
- Targeted Exploitation:** A strategic shift towards precision attacks using manufacturer-specific vulnerabilities, moving away from indiscriminate mass-scanning approaches.
- The Future:** Increasing sophistication through Mirai-Driven Botnets [4].

Future Research:

- Dynamic analysis frameworks
- Specialized detection technologies [4]

References

- M. Antonakakis et al., "Understanding the Mirai Botnet," *USENIX Security*, 2017.
- A. Marzano et al., "The evolution of bashlite and mirai IoT botnets," *IEEE S&P*, 2018.
- G. Vishwakarma and G. S. Chhabra, "Targeted DDoS Attacks on IoT: A Survey of the Mirai Botnet and its Variants," *AISC*, 2023.
- A. D. Alghazzawi et al., "An Efficient Hybrid Deep Learning Model for Detecting IoT Botnet Attacks," *IEEE Access*, 2024.
- Statista, 'IoT connected devices installed base worldwide,' 2024.

Research Paper Access

